Audspect BAS — Build Guide

Classification: Internal — Audspect Engineering / Confidential
Platform Version: v1.7.5

Overview

The Audspect BAS build pipeline runs on Windows (build host) via `packaging/windows-build.ps1` and produces:

- A garble-obfuscated Go orchestrator Docker image (garble runs *inside* the Docker build, not as a separate host-side step)
- A custom Caldera image with the CTID adversary-emulation library baked in
- RSA-4096-signed scenario YAML files, signed with an in-house signing tool (`orchestrator/scripts/signer.go`) — **not** GPG
- An RSA-4096-signed `BINARIES.sha256` agent-binary trust manifest, extracted from the built Docker image
- A versioned delivery folder/ZIP (`dist/bas-install-<version>.zip`) containing everything needed for an air-gapped install
- Optionally, a GPG-signed bundle checksum (`.zip.asc`) for transport integrity — a completely separate signing key from the RSA scenario/manifest signing above

The build host is the Windows machine where `packaging/windows-build.ps1` lives.

Prerequisites

Windows Build Host

Tool	Version	Install	Required?
Go	1.22+	https://go.dev/dl/	Yes — runs `signer.go`, `licensegen`, and native agent builds
Docker Desktop	latest	https://www.docker.com/products/docker-desktop	Yes
PowerShell	5.1+	Built into Windows	Yes
GnuPG (GPG)	2.x (Gpg4win preferred over Git-for-Windows' bundled MSYS gpg)	Winget: `winget install Gpg4win.Gpg4win`	Optional — bundle ships unsigned with a warning if missing

Garble is not a host-side prerequisite. It's installed and run entirely inside `orchestrator/Dockerfile`'s build stage (`GOGARBLE`-scoped to the orchestrator module) — nothing to install on the Windows host for it.

RSA Scenario/Manifest Signing Key

The RSA-4096 keypair used to sign scenario YAML files and `BINARIES.sha256` is generated automatically on first run of `windows-build.ps1` (`orchestrator/private_key.pem`), via `go run orchestrator/scripts/signer.go keygen`. The matching public key is compiled directly into the orchestrator binary (`orchestrator/internal/integrity/signing.go`) for runtime verification. Subsequent builds reuse the existing key.

Back up `orchestrator/private_key.pem` — losing it means scenarios and the binary manifest can never be validly re-signed for that install lineage.

Signature gate. Both release paths (`windows-build.ps1`, `build.sh`) run `go run scripts/signer.go verify-all <scenarios dir>` (from `orchestrator/`), which checks every `scenarios/**/*.yaml` (including `detection-profiles/` and `endpoint-mastery/`) with the binary's own `integrity.VerifyScenarioFile` against the compiled public key. A stale, tampered or missing `.sig`, or a placeholder (`SIGNING_KEYGEN_REQUIRED`) key, fails the build. Editing any scenario YAML therefore requires re-signing on the build host (`go run scripts/signer.go sign <abs path to orchestrator/private_key.pem> <yaml>`) and committing the updated `.sig` before release. `windows-build.ps1` re-signs automatically before the gate; `build.sh` does not. The gate also runs in `packaging/airgap/pack.sh`. `scenarios/custom/` and `scenarios/intel/` (directly under the root) are skipped, mirroring production, where they are unsigned by design. On `windows-build.ps1` everything is re-signed first, so the gate there mainly catches a key mismatch between `private_key.pem` and the compiled public key.

GPG Bundle-Signing Key

A separate key from the RSA one above — used only to sign the final delivery ZIP's checksum for transport integrity (`releases@audspect.com`). See Signing Infrastructure for key management. If GPG or the key isn't available, the build continues and warns that the bundle ships unsigned — it does not fail the build.

Running the Build

```
cd C:\path\to\Audspect_Cloud
.\packaging\windows-build.ps1 -Version 1.7.5 -Customer "C
```

Pass `-SkipBuild` to repackage the delivery bundle without rebuilding the Docker image (e.g. re-cutting a license or ZIP after a fix that didn't touch the image).

The script runs in numbered steps:

Step 0 — Verify Prerequisites

Confirms Docker is installed and the daemon is running, and that Go is installed.

Step 0b — Content Signing (Pre-Build)

Runs **before** the Docker build so signed artifacts travel into the image via the build context:

1. Generates the RSA-4096 keypair if `orchestrator/private_key.pem` doesn't exist yet (idempotent — skipped on later runs)
2. Signs every scenario YAML under `scenarios/` with `signer.go sign`, producing `.yaml.sig` files
3. Computes the SHA-256 hash of `orchestrator/wwwroot/index.html` — injected into the image via `--build-arg BAS_WWWROOT_HASH` so `StaticHandler()` halts at startup if the shipped dashboard doesn't match what was signed (UI tamper detection). Missing `index.html` disables this check with a warning, it does not fail the build.
4. `BINARIES.sha256` itself is signed later, in Step 5c — it's generated inside the Docker image, so it can't be signed until the image exists.

Step 1 — Build Docker Image (Orchestrator)

```
docker build -t "bas-orchestrator:$Version" --build-arg B
```

Garble (`-literals -tiny`, GOGARBLE-scoped to the orchestrator

module) runs inside this Docker build's build stage — see `orchestrator/Dockerfile` for the exact invocation. **Garble breaks Go's `html/template` field-name reflection.** The orchestrator uses JSON-tag-based map rendering for all report templates — never pass structs with field names directly to a template.

Step 2 — Pull/Build Dependency Images

```
docker pull postgres:16-alpine
docker pull ghcr.io/mitre/caldera:latest
docker pull chromedp/headless-shell:latest   # HTML→PDF r
docker build -t "bas-caldera:$Version" packaging\caldera
```

`bas-caldera:$Version` is a custom image with the CTID adversary-emulation library baked in (the only place that library is cloned — this build host needs internet access for it). If any of these fail, the build warns and continues with a reduced bundle (e.g. stock Caldera, or PDF reports falling back to the built-in renderer) rather than aborting.

Step 3-4 — Stage Delivery Folder and Save Images

Creates `dist\bas-install-<version>\` and saves every image as a `.tar` under `dist\bas-install-<version>\images\` (`docker load` accepts both `.tar` and `.tar.gz`).

Step 5a-5c — Build Agent Binaries

```
# 1. Agent binary embedded in the installer (-H windowsgu
#    Windows auto-allocates a visible console whenever so
#    the agent without an inherited console, e.g. the tra
#    at logon -- see the comment at this build line for t
go build -ldflags="-s -w -H windowsgui" -o installer\bas_

# 2. The installer EXE itself (embeds the agent binary ab
#    this is the actual customer-facing deliverable, down
#    dashboard's Agents -> Download Installer button
```

```
go build -ldflags="-s -w -H windowsgui" -o "$OutDir\BASAg

# 3. Standalone Windows agent (manual / side-by-side depl
go build -ldflags="-s -w -H windowsgui" -o "$OutDir\bas-a

# 4. Linux agent (amd64 + arm64)
$env:GOOS = "linux"; $env:GOARCH = "amd64"; $env:CGO_ENAB
go build -o agent\bas-agent-linux-amd64 .\agent\...
$env:GOOS = "linux"; $env:GOARCH = "arm64"; $env:CGO_ENAB
go build -o agent\bas-agent-linux-arm64 .\agent\...
```

The installer EXE's UAC manifest (`requireAdministrator`) and icon are embedded via `rsrc` (`github.com/akavel/rsrc`) before each Windows build, so elevation happens before the process starts rather than via a self-elevation dance. The build enforces a size guardrail on the installer EXE (errors above 25MB, expected ~14MB) to catch an accidental `go:embed` regression pulling in something that shouldn't be bundled.

There is currently no macOS agent build step in the pipeline, despite macOS-specific agent source files existing in the repo (`service_darwin.go`, `sysinfo_darwin.go`, etc.) — macOS is not cross-compiled or shipped in the delivery bundle today.

`BINARIES.sha256` is **not** computed by hashing local binaries — it's generated inside the Dockerfile's agent-builder stage (from the same binaries endpoints actually download) and extracted from the built image via `docker create` + `docker cp` (distroless has no shell, so this filesystem-level extraction is the only option). It's then RSA-signed with the same `private_key.pem` from Step 0b, staged into `orchestrator/agents/`, and copied into the delivery bundle.

Step 6-7 — Copy Bundle Files, Write VERSION

Copies scenarios (with `.yaml.sig` files), the dashboard `wwwroot`, and ART external payloads into the delivery folder, and writes a plain `VERSION` file containing the version string.

Step 7b — Air-Gap Integrity Manifest

Generates `verify.sh` and a per-file SHA-256 manifest so the bundle's own contents are independently verifiable after transfer, before install.

Step 8 — Generate License

```
go run packaging\licensing\licensegen\main.go -customer $
```

Copies the license into the delivery folder both as `<CustomerID>.lic` and as `bas.lic` (the `docker-compose.yml` volume mount name — without this exact filename, Docker creates an empty directory at that mount path and the license check fails). Skipped with a warning if `-Customer`/`-CustomerID` weren't passed, or if the licensing private key is missing.

Step 9 — Create Delivery ZIP

```
Compress-Archive -Path $OutDir -DestinationPath "dist\bas
```

Also writes a `.zip.sha256` bundle-level checksum file so the transfer itself is verifiable before unzip.

Step 9b — GPG-Sign the Bundle

Signs the ZIP with the separate GPG bundle-signing key (`releases@audspect.com`), producing `bas-install-<version>.zip.asc`, and stages a self-contained verify kit (`pubkey.asc` + `verify-sig.sh`) in `dist\` so the client can authenticate the ZIP before unzipping. Skipped gracefully (with a warning, not a build failure) if GPG or the key isn't available on this host.

Build Artifacts

After a successful build, under `dist\bas-install-<version>\`:

File	Description
`images/*.tar`	Docker image tarballs (orchestrator, Caldera, Postgres, headless-shell)
`BASAgent-Setup-<version>.exe`	Windows installer EXE — the customer-facing deliverable
`bas-agent-windows-amd64.exe`	Standalone Windows agent (manual/side-by-side deploy)
`agent/bas-agent-linux-amd64` / `-arm64`	Linux agent binaries
`agents/BINARIES.sha256` + `.sig`	RSA-signed agent binary trust manifest
`scenarios/*.yaml.sig`	RSA-4096 scenario signatures
`wwwroot/`	Dashboard static files (hash-checked at startup)
`<CustomerID>.lic` / `bas.lic`	Customer license
`VERSION`	Plain version string
`install.sh`, `docker-compose.yml`, `setup.sh`	Deployment scripts

At `dist\` root: `bas-install-<version>.zip`, `.zip.sha256`, and (if GPG signing succeeded) `.zip.asc` + a verify kit.

Common Build Failures

Error	Cause	Fix

`RSA keygen failed`	Go not installed, or `orchestrator/scripts/signer.go` missing	Verify Go install; verify the repo checkout is complete
`gpg: no suitable key found` / bundle ships unsigned	Signing key not imported, or GPG not installed	Import key: `gpg --import audspect-signing-key.asc`; see Signing Infrastructure
`html/template: field not found`	Struct used in template after garble	Use JSON-tag map rendering; never template struct fields
`docker: no space left on device`	Old images/build cache filling disk	`docker builder prune -a -f && docker image prune -f`
`error running keyboxd` (GPG)	Git-for-Windows' MSYS gpg's keyboxd not launchable from PowerShell	Prefer a full Gpg4win install over Git-for-Windows' bundled gpg; or remove `use-keyboxd` from `%USERPROFILE%\.gnupg\common.conf`
Agent binary trust all yellow after upgrade	`BINARIES.sha256` not refreshed for the new agent build	Re-run the full pipeline (not `-SkipBuild`) so Step 5c re-extracts and re-signs it

---

Running Tests

```
cd orchestrator
go test ./... -p 2              # Windows/Docker Desktop de
go test ./... -race             # Race detector
go test -cover ./...            # Coverage
```

Handler tests use `httptest.NewRecorder()`/`httptest.NewRequest()`. Database-dependent tests spin up a real PostgreSQL instance via `testcontainers-go` (Docker must be running) — there is no mock database, and every package spins up its own container. Pass `-short` to skip container-backed tests when Docker isn't available.

Windows dev host memory note: running `go test ./...` with no `-p` flag defaults to Go's full CPU-count parallelism, which on a memory-constrained Docker Desktop host (this build host has ~7.8GB total RAM, Docker capped at ~3.8GB) starts far too many Postgres containers at once and fails with spurious `unexpected EOF` Postgres-connection errors — not a real test/code defect. `-p 2` is the validated safe ceiling on this host (confirmed clean under a full cold run, ~35-45% faster than fully serial), but only once enough memory is free — stop `gopls.exe` first (`taskkill /F /IM gopls.exe`; it restarts automatically, no data lost) if free RAM is under ~1.5GB. If `-p 2` still throws `unexpected EOF` errors, fall back to `go test ./... -p 1` (fully serial, always safe, just slower).

© Audspect Engineering — Internal / Confidential